

Hidden-layer configurations in reinforcement learning models for stock portfolio optimization

Patrick Kevin Aritonang^{*}, Sudarso Kaderi Wiryono, Taufik Faturohman

School of Business and Management, Institut Teknologi Bandung, Indonesia

ARTICLE INFO

Keywords:

Reinforcement learning
Stock portfolio
Algorithmic trading
Financial market
Stock return
Neural network

ABSTRACT

In the rapidly evolving field of artificial intelligence and financial markets, efficient and adaptive portfolio management strategies are becoming increasingly critical. This study explores the impact of hidden-layer configurations in reinforcement learning models for stock portfolio optimization. Using a portfolio of 45 actively traded stocks in the Indonesian stock market, the performance of four reinforcement learning algorithms—Advantage Actor-Critic (A2C), Deep Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), and Twin Delayed Deep Deterministic Policy Gradient (TD3)—is evaluated with zero, one, and two hidden layers. The results show that A2C and DDPG models perform effectively without hidden layers, with A2C achieving the highest Cumulative Return (CuR) and Annualized Return Rate (ARR) among all configurations. Adding hidden layers to A2C improved risk management, resulting in a lower Maximum Drawdown (MDD) and a higher Annualized Sharpe Ratio (ASR). DDPG exhibited consistently strong performance, with its zero hidden-layer model showing the highest ASR. Conversely, PPO underperformed across all configurations, with negative returns in the zero-layer setup and marginal improvements with added complexity. Introducing additional hidden layers improved TD3's performance, enhancing risk-adjusted returns. These findings suggest that the effectiveness of hidden-layer configurations depends on the specific algorithm used. While A2C and DDPG benefit from increased complexity, simpler architectures may be more suitable for PPO and TD3. This study offers new insights into optimizing reinforcement learning models for stock portfolio management by adjusting hidden-layer structures to balance returns and risk.

1. Introduction

The implementation of artificial intelligence (AI) in the financial market has revolutionized how investors and institutions approach trading, risk management, and portfolio optimization. A study emphasizes how reinforcement learning, integrated with generative models, can significantly improve portfolio management by enhancing the generalization capabilities of AI models, leading to better investment decisions (Kuo et al., 2021). Research on artificial intelligence applied to stock market trading reviews the extensive use of AI in price forecasting, portfolio optimization, and sentiment analysis, highlighting the continuous advancements and increasing specificity of AI applications in the financial sector (Ferreira et al., 2021). As the financial market rapidly changes, AI use is becoming increasingly prominent.

Reinforcement learning (RL), a subset of AI, has emerged as a compelling approach for portfolio optimization. Extensive study on RL algorithms for dynamic asset allocation, showing that RL outperforms

both passive and active trading strategies by utilizing advanced techniques like cross-sectional analysis in addition to time series analysis (Aboussalah et al., 2020). Alonso and Srivastava (2020) highlighted that deep reinforcement learning (DRL) significantly outperforms traditional portfolio management approaches by effectively managing transaction costs and optimizing asset allocations using time series data and neural networks (Alonso & Srivastava, 2020). Unlike other machine learning techniques that need a static dataset, RL involves an agent that learns to make decisions by actively interacting with the market environment, continuously refining its strategy to maximize cumulative rewards.

This study explores a new dimension of neural network architecture in portfolio management by examining the role of hidden-layer configurations. Researchers have extensively studied the addition of hidden layers in neural networks for stock price and return forecasting, demonstrating that deeper architectures improve predictive accuracy (Abolmakarem et al., 2024; Althelaya et al., 2021; Zhang & Shen, 2019), the role of deep neural networks (DNNs) in enhancing accuracy and

^{*} Corresponding author.

E-mail address: patrick.aritonang@sbm-itb.ac.id (P.K. Aritonang).

<https://doi.org/10.1016/j.iswa.2024.200467>

Received 17 September 2024; Received in revised form 2 December 2024; Accepted 15 December 2024

Available online 16 December 2024

2667-3053/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

computational efficiency is evident in trajectory optimization problems, as demonstrated by (Mughal et al., 2022). However, limited research has explored how hidden-layer configurations affect portfolio management. Portfolio management requires balancing return maximization and risk management, making it a more complex and dynamic than single-asset forecasting. This study seeks to address this gap by examining how different hidden-layer configurations affect the performance of reinforcement learning models in portfolio optimization. This study aims to investigate the following key research questions.

RQ1: How does the number of hidden layers in RL models affect portfolio performance?

RQ2: Which Reinforcement Learning algorithms—Advantage Actor-Critic (A2C), Deep

Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), or Twin

Delayed Deep Deterministic Policy Gradient (TD3)—perform best with different hidden layer configurations?

This research aims to enhance portfolio optimization strategies by identifying the most effective RL model layer architectures, providing valuable insights for academic research and practical investment management

1.1. The emergence of reinforcement learning

Reinforcement learning (RL) is a branch of machine learning that focuses on how agents should act in an environment to maximize cumulative rewards. Reinforcement learning methods such as deep deterministic policy gradient (DDPG) and their application in complex control tasks, demonstrating their capability in continuous action spaces, are thoroughly reviewed (Tan, 2021). Applying deep reinforcement learning in trading strategies, mainly using algorithms like deep Q-learning and advantage actor-critic (A2C), underlines RL's utility in highly dynamic and uncertain environments (Sagiraju & Shashi, 2021). In real-world applications such as robotics, reinforcement learning has been successfully implemented to ensure safe and reliable operation, even in environments with unforeseen perturbations, by defining conditions that guarantee recovery to safety and proposing strategies for efficient recovery within finite steps (Sreenivas & Rao, 2022). Reinforcement learning has been utilized for real-time trajectory planning in satellite orbit transfers, achieving high accuracy in predicting transfer times by leveraging deep neural networks (DNNs) for Q-value estimation (Hassaan et al., 2022). Researchers have highlighted the importance and applications of RL algorithms like Q-learning, deep Q-networks, and actor-critic methods in various environments, showing their effectiveness in production planning and control (Esteso et al., 2022). Unlike supervised learning, which relies on labelled datasets, reinforcement learning works well in dynamic and unpredictable market conditions, making it more suitable for trading decisions requiring sequential actions.

Fig. 1 illustrates the fundamental structure of a reinforcement learning (RL) system, which involves the interaction between an agent and its environment. The agent uses this reward to evaluate its action and adjust the trading behaviour to maximize the cumulative return. This cyclical process continues state observation and action selection.

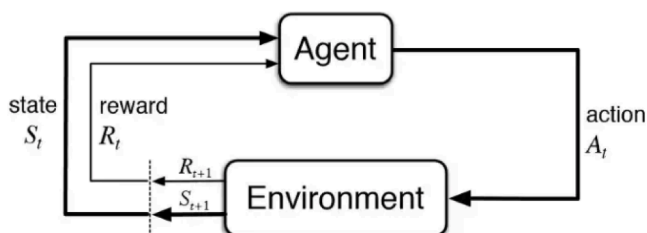


Fig. 1. Basic Framework of Reinforcement Learning.

1.2. Reinforcement learning for financial portfolio optimization

The application of RL to financial asset portfolios has gained significant attention in recent years, as traditional approaches such as the Markowitz mean-variance framework, often fail to account for the volatility nature of financial markets. A novel RL-based system for portfolio management has outperformed traditional constant rebalanced portfolios, proving the scalability and reusability of RL in financial markets (Huang & Tanaka, 2021). RL models significantly improve portfolio allocations by accounting for asset variability and correlation better than traditional methods (Olschewski et al., 2021). A deep RL-based approach to portfolio management achieves over 24 % average daily returns, surpassing state-of-the-art methods (Baca & Chen, 2021). Incorporating RL into investment portfolios can increase returns by 12 % without a commensurate rise in risk, outperforming traditional strategies (Santos et al., 2023). The use of deep RL for optimizing high-frequency bitcoin transactions has shown a 31.67 % improvement in returns over traditional benchmarks (Liu et al., 2021). DRL models can dynamically learn temporal and spatial dependencies between assets, optimizing portfolios for low correlation to enhance risk management while ensuring returns (Bai et al., 2023; Jiang & Wang, 2022). Reinforcement learning with heavy-tailed preserving normalizing flows can simulate the high-dimensional joint probability of financial markets, improving portfolio optimization and mitigating the impact of unpredictable market crises (Huang et al., 2023). Advanced RL models, like Transformer Actor-Critic with Regularization (TACR), incorporate past market data and attention networks to improve decision-making and performance metrics like the Sharpe ratio (Lee & Moon, 2023). A recent study proposes a novel reinforcement learning-based shrinkage estimator for large covariance matrices in high-dimensional portfolio selection, demonstrating that a Recurrent Policy Gradient Agent (RPGA) provides superior out-of-sample performance in asset allocation (Mattera and Mattera, 2023). RL offers a more flexible and adaptive approach to learning and adjusting strategies in real time.

However, RL models come with limitations that need to be addressed for effective implementation in financial markets; many RL models in finance ignore or underestimate the impact of transaction costs, liquidity constraints, and bid-ask spreads. These factors affect the profitability of trading strategies, often leading to overestimating expected returns in simulated environments (Meng & Khushi, 2019). Applying RL to high-frequency trading or large-scale portfolio management presents scalability challenges. The computational complexity of RL models increases significantly with the size of the state and action spaces, making it challenging to apply these models effectively in real-time trading environments (Aboussalah et al., 2020). Defining appropriate reward functions that accurately reflect investors' goals (e.g., risk-adjusted returns) takes time in financial applications. Poorly designed reward functions can lead to suboptimal or harmful strategies (Goluzza et al., 2023). Addressing these challenges is crucial for successfully implementing RL in portfolio optimization.

1.3. FinRL-based Reinforcement learning

The paper "FinRL: Deep Reinforcement Learning Framework to Automate Trading in Quantitative Finance" prepared by Liu et al., 2020 presents a comprehensive framework designed to simplify the application of deep reinforcement learning (DRL) in quantitative finance. FinRL is designed as an open-source complete pipeline framework to ease the steep learning curve associated with DRL in financial markets. It provides a three-layer architecture with state-of-the-art DRL algorithms, fine-tuned reward functions, and modular structures to facilitate debugging and customization (Liu et al., 2020). The FinRL library allows beginners to get hands-on experience in quantitative finance, supporting the development of custom trading strategies with pre-configured environments and extensive back-testing tools. It emphasizes ease of use with tutorials, reproducibility, and adaptability to various trading

scenarios (Liu et al., 2020).

FinRL-Podracar, an extension of the FinRL framework, focuses on high performance and scalability. It incorporates a generational evolution mechanism to improve DRL agent performance and optimizes training on GPU clouds, significantly reducing training time while enhancing trading performance (Liu et al., 2020). FinRL-Meta builds a universe of market environments to support the complex demands of DRL in financial simulations. It separates data processing from the strategy design pipeline and provides a wide range of market environments for diverse trading tasks, supporting extensive simulation and multiprocessing capabilities (Liu et al., 2020).

Based on Fig. 2. The FinRL framework organizes its structure into three layers: the environment layer, the agent layer, and the application layer. The environment layer simulates financial markets using historical data and live trading APIs, providing a realistic training ground for DRL agents. The agent layer implements various state-of-the-art DRL algorithms, allowing users to experiment with different strategies and configurations. The application layer ties these components together, enabling users to deploy and evaluate their strategies in real-world scenarios. The framework's emphasis on reproducibility and hands-on tutorials makes it accessible to users with different levels of expertise, fostering the development of robust and effective trading strategies.

The FinRL framework, with its comprehensive architecture, is a powerful tool for researchers and practitioners alike. It enables them to explore and implement advanced reinforcement learning strategies in various financial contexts.

2. Research framework and methods

2.1. The framework of reinforcement learning with multi-hidden layers

Refer to Fig. 3; the framework for reinforcement learning (RL) with multi-hidden layers is designed to enhance the decision-making capabilities of trading agents in complex financial environments. This framework integrates two hidden layers within each RL algorithm, including A2C, DDPG, PPO, and TD3, to better capture and process the intricate patterns and dynamics of the market. The hidden layers act as

intermediaries that transform the input state, including key market indicators like open, high, low, and close prices and volume, into a more abstract representation before the agent decides.

By employing this multi-layered approach, the framework aims to improve the agent's ability to learn and adapt to market changes, ultimately leading to more effective portfolio optimization strategies. Subchapter 3 provide more detailed explanations of each component of this framework, including the environment, state, trading agents, actions, and rewards.

2.2. Environment

The environment in this research refers to the simulated financial market in which the reinforcement learning agents operate. Specifically, the environment is constructed using a portfolio of 45 stocks that are part of the LQ45 Index, representing the most liquid and actively traded stocks on the Indonesia Stock Exchange, accounting for 62.9 % of the market capitalization as of December 2023. The Indonesian stock market's resilience and performance during global economic uncertainties underscore its significance in the broader financial system (Hashmi et al., 2021). The Indonesian stock market has strategically mobilized private investment and developed private enterprise in Indonesia (Eng., 2022). The Indonesian stock market has grown significantly, driven by positive macroeconomic factors like GDP and foreign portfolio investments (Siska et al., 2023). This setup allows for a realistic evaluation of the agents' performance in managing a significant and diverse portfolio within the Indonesian financial market.

2.3. State

The state represents the set of input features that describe the current condition of the financial environment at any given time. The state in this study includes daily market data of the open price, high price, low price, close price, and trading volume for each of the 45 stocks in the LQ45 Index. These features are updated regularly to reflect real-time market conditions. Technical analysis of high, low, and close prices provides valuable insights into market behavior, aiding in portfolio

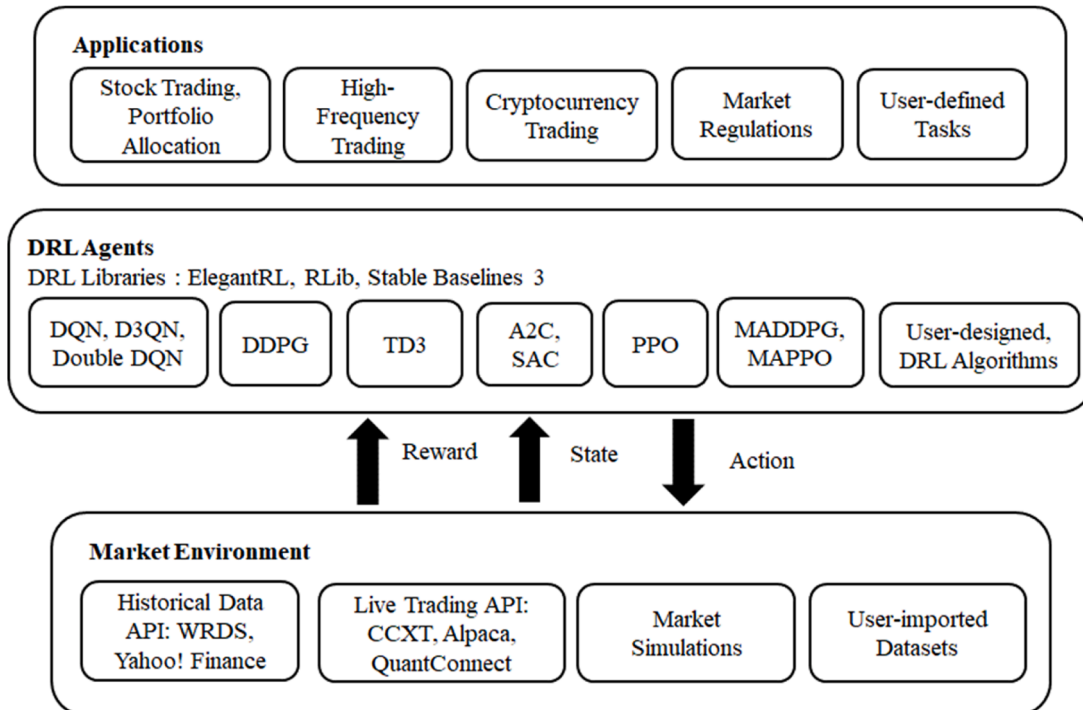


Fig. 2. Overview of FinRL (Liu et al., 2020).

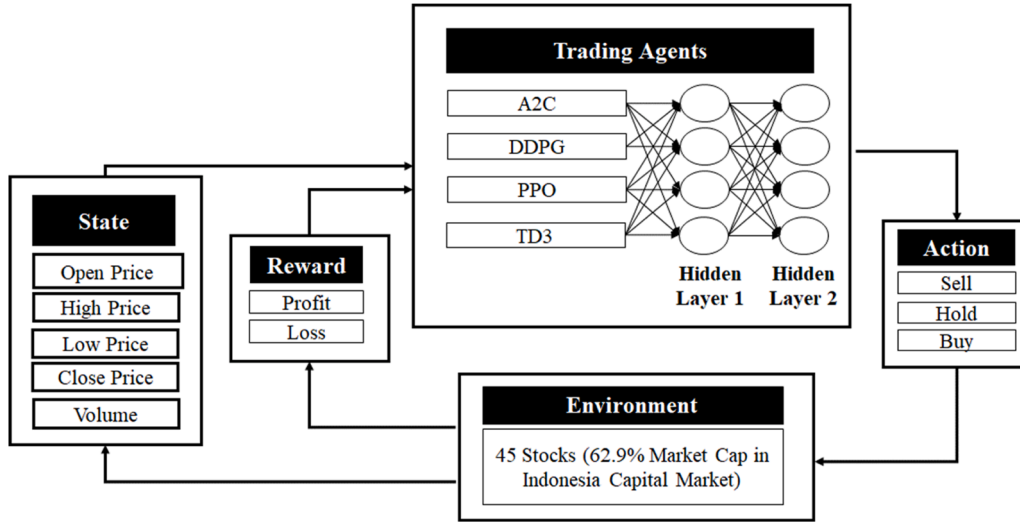


Fig. 3. Framework of the Multi Hidden-Layer Reinforcement Learning (own work., 2024).

management and optimization (Fiess & MacDonald, 2002). A study highlights that low price, high price, and close price significantly contribute to portfolio optimization by improving trading strategies, especially in volatile markets (Weng et al., 2020). Incorporating high, low, open, and volume prices into prediction models enhances forecasting accuracy, reducing losses compared to traditional models (Alkhatib et al., 2022). By incorporating these specific market indicators, the state effectively captures the dynamic nature of the financial environment, allowing the agents to learn and adapt their trading strategies in response to changing market conditions.

2.4. Trading agents

The trading agents are responsible for making decisions that optimize portfolio performance. Each trading agent in this study is based on a different reinforcement learning algorithm: Advantage Actor-Critic (A2C), Deep Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), and Twin Delayed Deep Deterministic Policy Gradient (TD3). DDPG and PPO have shown notable success in stock portfolio optimization by effectively managing transaction costs and risk factors in continuous action spaces (Hieu, 2020). A study combining long- and short-term risk control with TD3 demonstrated superior performance in portfolio management, offering better returns and risk mitigation compared to standard approaches (Jiang & Wang, 2022). Research demonstrates that an ensemble of actor-critic algorithms, including A2C, DDPG, PPO, and TD3, can enhance trading performance by leveraging the strengths of each approach to navigate complex market environments (Kong & So, 2023). This research chooses these algorithms for their effectiveness in handling continuous action spaces and their capability to learn complex trading strategies.

This research enhances each agent with two hidden layers, enabling more capability in processing input data and allowing the agents to capture complex market patterns. Adding hidden layers improves the performance of hybrid neural networks in predicting stock market trends, mainly when dealing with complex time-series data (Hao & Gao, 2020). Deep neural networks with multiple hidden layers significantly enhance the prediction accuracy of stock market indices when combined with metaheuristic techniques (Rath et al., 2021). More complex neural network structures with additional hidden layers significantly improve accuracy in stock market forecasting. (Fan & Peng, 2022). Multiple algorithms provide a comprehensive comparison of their performance, helping to identify which approach is most effective for portfolio optimization in the context of the LQ45 Index.

2.5. Action

In the reinforcement learning framework, the action represents the decision made by the trading agents at each time step, based on the current state of the environment. For this study, the actions available to the agents are threefold: Buy, Hold, or Sell any of the 45 stocks included in the LQ45 Index. These actions determine how the agents adjust the portfolio in response to the observed market conditions.

When the agent selects the Buy action, it increases the positioning in a particular stock, allocating more capital to it. Conversely, choosing the Sell action decreases the position, potentially freeing up capital or mitigating losses. The Hold action indicates that the agent maintains the current position without changing it.

2.6. Reward

In this study, the reward is primarily derived from the profit and loss in the trading, which contributes to the performance of the portfolio assessed using several key financial metrics: Total Asset Value (TAV), Annualized Return Rate (ARR), Cumulative Return (CuR), Maximum Drawdown (MDD), and Annualized Sharpe Ratio (ASR). It is defined as follows:

$$\text{Total Asset Value (TAV)}_t = \sum_{i=1}^n p_{i,t} \times q_{i,t} + \text{Cash}_t \quad (1)$$

Where $p_{i,t}$ is the price of asset i at time t . $q_{i,t}$ is the quantity of asset i at time t . Cash_t is the cash balance at time t .

$$\text{Annualized Return Rate (ARR)} = \left(\frac{\text{TAV}_{\text{end}}}{\text{TAV}_{\text{start}}} \right)^T - 1 \quad (2)$$

$$\text{Cumulative Return (Cur)} = \frac{\text{TAV}_{\text{end}} - \text{TAV}_{\text{start}}}{\text{TAV}_{\text{start}}} \quad (3)$$

$\text{TAV}_{\text{start}}$ and TAV_{end} are the total asset values at the start and end of the period.

$$\text{Maximum Drawdown (MDD)} = \min \left(\frac{\text{TAV}_t - \text{TAV}_{\text{peak}}}{\text{TAV}_{\text{peak}}} \right) \quad (4)$$

TAV_t is the total asset value at time t . TAV_{peak} is the peak total asset value prior to time t .

$$\text{Annualized Sharpe Ratio (ASR)} = \frac{E(R_p) - R_f}{\sigma_p} \quad (5)$$

$E(R_p)$ is the expected return of the portfolio, R_f is a risk-free rate, and σ_p is the standard deviation of the portfolio's returns (volatility).

All the above formulas will be used to evaluate and compare the performance of the different reinforcement learning models applied in this study. These metrics provide a comprehensive assessment of the portfolio's return, risk, and risk-adjusted return, allowing for a thorough analysis of how each model configuration impacts overall investment outcomes.

3. Training algorithm and hyperparameter

In this chapter, we outline the comprehensive process for training reinforcement learning (RL) agents, as shown in Fig. 4, using a unified algorithmic framework and its hyperparameter.

3.1. Model initialization

This involves initializing the model components. First, the appropriate RL agent is selected from A2C, DDPG, PPO, or TD3 based on the specific requirements of the task. Next, the policy network (Actor) is initialized, which guides the agent's actions and may include one or two hidden layers depending on the complexity required. The value network (Critic) is also initialized, which estimates the value of states, and for TD3 and DDPG, two additional Critic networks are set up to improve training stability. The process concludes with the initialization of optimizers for both the policy and value networks to facilitate learning through gradient descent.

3.2. Training parameters initialization

During this step, the maximum number of episodes and time steps per episode are defined, along with the discount factor γ , which controls the consideration of future rewards. Learning rates for the Actor and Critic networks are set to determine the step size for updates. For DDPG, TD3, and PPO, a replay buffer is initialized to store experience tuples for future training, playing a crucial role in stabilizing the learning process. Additionally, any model-specific parameters, such as noise levels for

exploration in DDPG and TD3, are defined at this stage.

3.3. Training loop per episode

This step forms the core of the training process, running iteratively for each episode. At the beginning of each episode, the starting state s_0 is initialized. For each time step t within the episode, actions are selected. A2C and PPO sample actions using the policy network, which outputs a probability distribution over possible action. DDPG and TD3 sample continuous actions from the deterministic policy network, adding noise to encourage exploration. The chosen action is then executed, and the resulting reward r_t and next state s_{t+1} are observed. A2C and PPO store experience tuples $\langle s_t, a_t, r_t, s_{t+1} \rangle$ to compute advantages, while DDPG and TD3 store these tuples in the replay buffer for later use in training.

3.4. Computation of advantages and Q-Values

The advantage function or Q-values are calculated based on the agent's experience in the next critical step. A2C and PPO compute the advantage function $A(s_t, a_t)$ using the Critic's estimates, with PPO further computing target values for policy optimization using a clipping method to ensure stable updates. For DDPG and TD3, the target Q-value is computed using the Critic network, followed by updating the Critic based on the gradient of the Q-function.

3.5. Network updates

In this step, A2C updates the Critic by minimizing the temporal difference (TD) error and the Actor by maximizing the advantage. PPO performs updates using the clipped objective to maintain stability, updating both the value and policy networks. DDPG updates the Critic using the deterministic policy gradient and the Actor by maximizing the Q-value. TD3 applies delayed updates to both Critic networks and the Actor network to stabilize the training process further.

3.6. Exploration regularization

This step ensures sufficient exploration during training. A2C and PPO apply entropy regularization to encourage exploration of the action space, while DDPG and TD3 introduce action noise to maintain exploration throughout the training process.

3.7. Target network updates

This step is periodically performed using soft updates. The target networks are updated as $\theta' \leftarrow r\theta + (1 - r)\theta'$, where θ represents the current network parameters, and θ' represents the target network parameters.

3.8. Continuation of training and model evaluation

This step involves running the training process for the specified number of episodes or until the agent's performance converges to a satisfactory level. The performance of the trained agent is evaluated using validation or testing environments.

3.9. Experimental hyperparameter

Table 1 below presents the key configurations of environmental hyperparameters:

To ensure robustness and avoid overfitting, as shown in Table 1, where the model is trained on data from January 1, 2010, to April 30, 2022, and then tested on an out-of-sample period from May 1, 2022, to October 31, 2023. The hyperparameters defined above set the basic

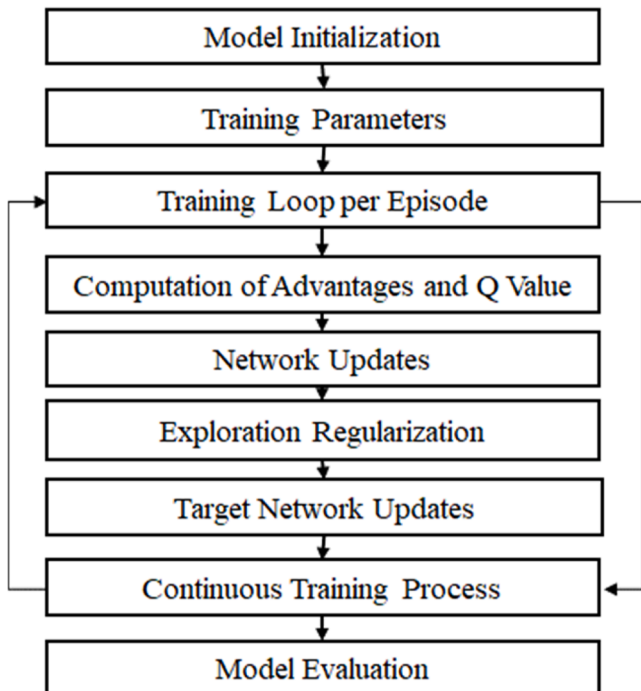


Fig. 4. Training Algorithm Steps (own work., 2024).

Table 1
Experimental hyperparameter.

Parameter	Value
Stock Dimensions	45
Action Space Size	45 (one action per stock)
State Space	451
Currency Trading	IDR
Initial Cash Available	IDR 100,000,000
Transaction Cost (Buying)	0.001 (0.1 %)
Training Period	01 January 2010 – 30 April 2022
Test Period	01 May 2022 – 31 October 2023
Transaction Cost (Selling)	0.001 (0.1 %)
Reward Scaling Factor	1.00E-04
Verbosity Level	1 (printing)
Time Steps	10,000

foundation for the training step, ensuring that the simulated trading mechanism is as close to real-world trading conditions as possible. The stock dimensions and action space size are particularly crucial, as they directly impact the complexity of the decision-making process that the agents must navigate.

3.10. Hyperparameter for models

Table 2 below presents the hyperparameters for each model; these parameters control various aspects of the training process, including how frequently the models update their networks, how they handle exploration, and how they process experience from interactions with the environment.

These hyperparameters are critical in shaping the behavior of the models during training. For instance, the learning rate directly impacts how quickly the models adjust to new information, while the entropy coefficient helps balance the trade-off between exploration and exploitation. Including zero, one, and two hidden layer policies allows for a comparative analysis of how network depth affects the performance of each model, providing insights into the optimal configurations for stock portfolio management. These hyperparameters' careful selection and tuning ensure that the models can learn from the complex, dynamic market environment and develop effective trading strategies.

Moreover, other parameters, such as the number of steps (*n_steps*) and batch size, influence how efficiently the models process training data. Parameters like *clip_range_vf* in PPO, which controls the value function clipping, are crucial for maintaining stable learning while avoiding excessive deviations in policy updates. Similarly, using *gae_lambda* in PPO helps with advantage estimation, ensuring the model

Table 2
Hyperparameter for models.

Model	Parameter	Value
Advantage Actor-Critic (A2C)	<i>n_steps</i>	5
	<i>ent_coef</i>	0.005
	<i>learning_rate</i>	0.0003
	<i>ent_coef</i>	0.01
	<i>n_steps</i>	4096
Proximal Policy Optimization (PPO), or	<i>learning_rate</i>	0.0001
	<i>batch_size</i>	128
	<i>n_epochs</i>	30
	<i>clip_range_vf</i>	0.2
	<i>gae_lambda</i>	0.8
Deep Deterministic Policy Gradient (DDPG)	<i>action_noise</i>	Gaussian
	<i>buffer_size</i>	10,000
	<i>learning_rate</i>	0.0002
	<i>batch_size</i>	128
	<i>learning_rate</i>	0.02
Twin Delayed Deep Deterministic Policy Gradient (TD3)	Policy	MlpPolicy
One Hidden Layer Policy	<i>policy_kwargs=dict(net_arch=[64])</i>	
Two Hidden Layer Policy	<i>policy_kwargs=dict(net_arch=[64, 64])</i>	

adequately balances future rewards with immediate returns. DDPG and TD3 utilize noise in their action space (Gaussian noise) to ensure proper exploration, especially in continuous action spaces, which is critical to improving performance in stock portfolio optimization.

To support reproducibility and allow further exploration of our findings, the data and code used in this study will be made publicly available to the scientific community through a GitHub repository following the publication of this article. The shared resources will include the implementation of the reinforcement learning models—Advantage Actor-Critic (A2C), Deep Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), and Twin Delayed Deep Deterministic Policy Gradient (TD3)—with various hidden-layer configurations.

3.11. Complexity of model configurations

The complexity of the proposed reinforcement learning models varies with the number of hidden layers, algorithm type, and the size of the action and state spaces. Adding hidden layers increases the parameters and computational demands, requiring more processing power and memory for backpropagation and training. Among the algorithms, policy gradient methods like Advantage Actor-Critic and Proximal Policy Optimization have relatively lower complexity due to batch updates, while Deep Deterministic Policy Gradient and Twin Delayed Deep Deterministic Policy Gradient incur higher complexity with continuous action spaces and the need for target networks to maintain stability.

3.12. Computational time and system specifications

To evaluate the efficiency of the methodology, we recorded the computational time required for each of the twelve model configurations. Each model took approximately seven hours to complete, resulting in a total computational time of 84 h. This research conducts the experiments on a computer with an M2 processor, 16GB of RAM, and a macOS operating system.

4. Result

4.1. Impact of hidden layer configuration on portfolio performance

Table 3 presents the financial performance metrics for each reinforcement learning model, configured with either zero (*m0*), one (*m1*), or two hidden layers (*m2*). The metrics include Cumulative Return (CuR), Annualized Return Rate (ARR), Maximum Drawdown (MDD), and Annualized Sharpe Ratio (ASR).

The results in Table 3 show that A2C and DDPG perform best with zero hidden layers. A2C with no hidden layers (*m0_A2C*) achieves the highest Cumulative Return (CuR) of 51.54 % and Annualized Return Rate (ARR) of 49.39 %, with a solid Annualized Sharpe Ratio (ASR) of 0.83. Similarly, DDPG without hidden layers (*m0_DDPG*) shows strong performance, with a CuR of 33.80 %, an ARR of 32.46 %, and the highest

Table 3
Portfolio Performance Comparison.

Model	CuR (%)	ARR (%)	MDD (%)	ASR (%)
<i>m0_A2C</i>	51.54	49.39	−35.70	0.83
<i>m0_DDPG</i>	33.80	32.46	−13.07	0.94
<i>m0_PPO</i>	−9.04	−8.75	−29.27	−0.05
<i>m0_TD3</i>	33.81	32.47	−13.06	0.94
<i>m1_A2C</i>	−40.87	−39.80	−66.71	−0.21
<i>m1_DDPG</i>	33.76	32.43	−13.07	0.93
<i>m1_PPO</i>	55.27	52.94	−35.73	0.87
<i>m1_TD3</i>	15.52	14.95	−33.45	0.46
<i>m2_A2C</i>	41.18	39.52	−13.31	1.14
<i>m2_DDPG</i>	13.29	12.80	−33.23	0.42
<i>m2_PPO</i>	−25.61	−24.85	−42.22	−0.32
<i>m2_TD3</i>	7.83	7.55	−35.67	0.31

ASR of 0.94, indicating its superior ability to manage risk and reward. In contrast, PPO with no hidden layers (m0_PPO) underperforms significantly, posting negative returns with a CuR of -9.04% and an ARR of -8.75% , suggesting it struggles without added complexity. TD3 performs moderately well in its simplest form (m0_TD3), delivering a CuR of 33.81% and an ARR of 32.47% , along with a respectable ASR of 0.94.

As hidden layers are introduced, the performance of A2C and PPO models changes drastically. A2C with one hidden layer (m1_A2C) suffers a substantial drop in performance, with a CuR of -40.87% and the lowest ASR of -0.21 among all configurations, indicating that additional complexity does not benefit A2C. PPO, however, shows significant improvement with one hidden layer (m1_PPO), achieving a CuR of 55.27% and an ARR of 52.94% , coupled with a strong ASR of 0.87. Interestingly, PPO's performance declines sharply with two hidden layers (m2_PPO), suggesting overfitting with more complexity, as evidenced by a negative CuR of -25.61% and an ASR of -0.32 . The TD3 model remains relatively stable across all configurations, but its best performance is observed with zero hidden layers. Overall, these results emphasize that the optimal hidden-layer configuration depends on the specific reinforcement learning algorithm, with A2C and DDPG performing best with simpler architectures. At the same time, PPO benefits from a balanced level of complexity.

Based on the results in Table 3, this study conducted a significance test using ANOVA to assess differences across performance metrics for the reinforcement learning models examined. The ANOVA results reveal statistically significant differences, with an F-value of 11.997, which exceeds the critical F-value of 2.816 at the 0.05 significance level. Additionally, the low p-value of $7.21\text{E-}06$ confirms the presence of meaningful variability among the Cumulative Return (CuR), Annualized Return Rate (ARR), Maximum Drawdown (MDD), and Annualized Sharpe Ratio (ASR) across the models. These findings suggest that the observed differences in returns and risk-adjusted performance are unlikely to be due to random variation, underscoring the importance of selecting appropriate model configurations to achieve optimal portfolio performance.

4.2. Total asset value (TAV) movement

The following graphs illustrate the movement of Total Asset Value (TAV) for each reinforcement learning model with both one and two hidden layers, starting from an initial investment of IDR 100,000,000.

The figures above illustrate the Total Asset Value (TAV) movements for four reinforcement learning models (A2C, DDPG, PPO, and TD3) across three different configurations: zero hidden layers (Fig. 5), one hidden layer (Fig. 6), and two hidden layers (Fig. 7). In Fig. 5, which shows the performance of models without hidden layers, the A2C model (blue line) performs exceptionally well, reaching a high TAV early in the period but showing signs of increased volatility towards the end. The

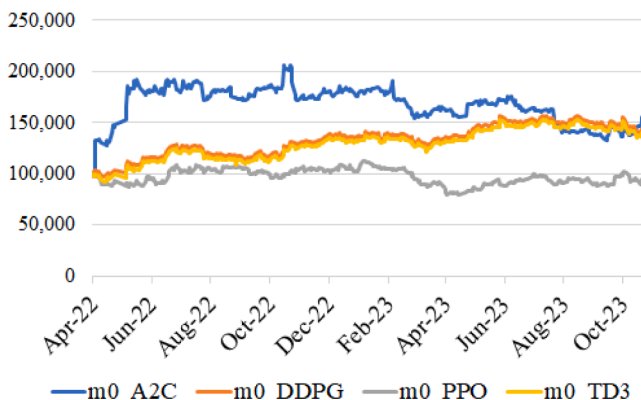


Fig. 5. TAV movement with zero hidden layer.

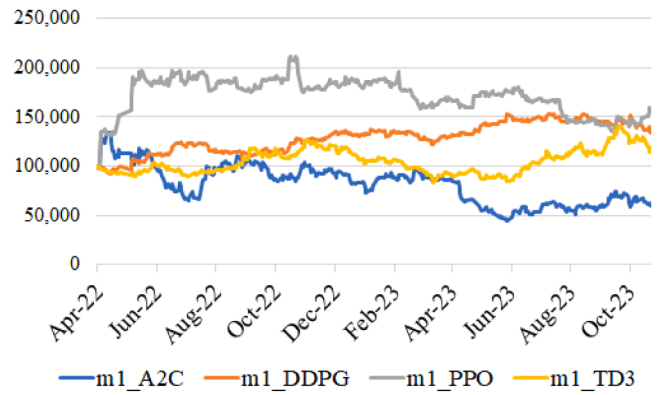


Fig. 6. TAV movement with one hidden layer.

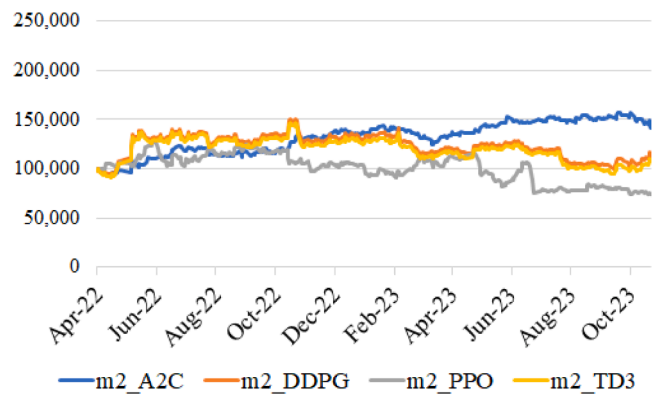


Fig. 7. TAV movement with two hidden layers.

DDPG (orange line) and PPO (grey line) models demonstrate more stable growth, with DDPG showing relatively consistent performance. TD3 (yellow line) exhibits moderate growth, maintaining a relatively stable trajectory.

In Fig. 6, introducing one hidden layer appears to negatively impact A2C's performance (blue line), which shows a steep decline in TAV. However, PPO (grey line) experienced significant improvement by adding a hidden layer, outperforming the other models, and achieving the highest TAV by the end of the period. DDPG (orange line) remains stable, while TD3 (yellow line) experiences minor fluctuations but generally maintains consistent performance. In Fig. 7, the two hidden-layer configurations highlight further improvements in A2C's stability (blue line), with the model recovering and showing gradual TAV growth. On the other hand, PPO (grey line) suffers from overfitting, with its TAV declining sharply. DDPG (orange line) and TD3 (yellow line) exhibit stable but unimpressive performance, showing that additional hidden layers do not significantly benefit these models. Overall, the figures highlight the varied impact of hidden layers on model performance, with some models benefiting from increased complexity while others perform better with simpler architectures.

The Figs. 8–11 illustrates the Total Asset Value (TAV) movements for each reinforcement learning model (A2C, DDPG, PPO, and TD3) across different hidden-layer configurations: zero, one, and two hidden layers. In Fig. 8, the A2C model demonstrates a clear preference for zero hidden layers (m0_A2C), as it consistently outperforms the other configurations, reaching a high TAV early in the period and maintaining a relatively stable performance. However, A2C with one hidden layer (m1_A2C) significantly underperforms, showing a steep decline in TAV, suggesting that added complexity does not benefit this model. The two hidden-layer configuration (m2_A2C) offers a middle ground, with modest improvement in stability compared to m1_A2C, though it does not surpass the

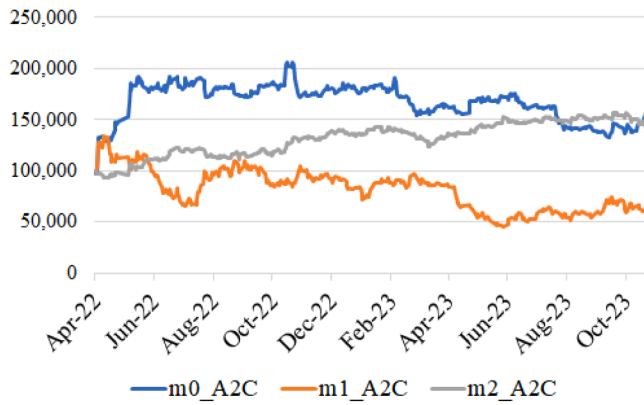


Fig. 8. TAV movement of A2C models.

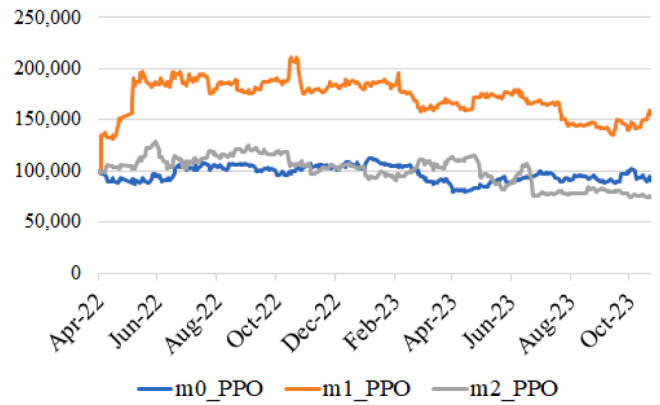


Fig. 10. TAV movement of PPO models.

zero-layer model.

In Fig. 9, the DDPG model shows consistent performance across all configurations, with m0_DDPG and m1_DDPG exhibiting similar growth patterns and relatively stable TAV. The m2_DDPG configuration underperforms slightly, indicating that adding complexity in hidden layers does not significantly enhance DDPG's performance. Figs. 10 and 11 highlight the performance of the PPO and TD3 models, respectively. The PPO model (Fig. 10) benefits significantly from the additional of one hidden layer (m1_PPO), achieving the highest TAV, while the zero hidden-layer model (m0_PPO) struggles with lower returns. However, the two hidden-layer configuration (m2_PPO) results in a decline in performance, indicating potential overfitting. TD3 (Fig. 11) exhibits stable growth across all configurations. However, the one hidden-layer model (m1_TD3) shows slight underperformance compared to the other configurations, suggesting that TD3 is relatively robust to variations in model complexity.

4.3. Unique finding related to hidden-layer configurations

In stock price or return forecasting tasks, adding hidden neural layers often improves the model's ability to capture complex, non-linear relationships in the data, leading to more accurate predictions. Zhang and Shen (2019) observed that a multi-layered neural network outperformed single-layer models (Zhang & Shen, 2019). Using multiple hidden layers in DNNs enhances the model's ability to learn intricate patterns, leading to higher prediction accuracy (Khaled et al., 2021). Studies employing multi-layer perceptrons (MLP) for stock price prediction have shown improved predictive performance with additional layers, as they capture non-linear market behaviors more effectively (Abolmakarem et al., 2024). However, in the case of portfolio management, our results indicate that adding hidden layers only sometimes leads to better

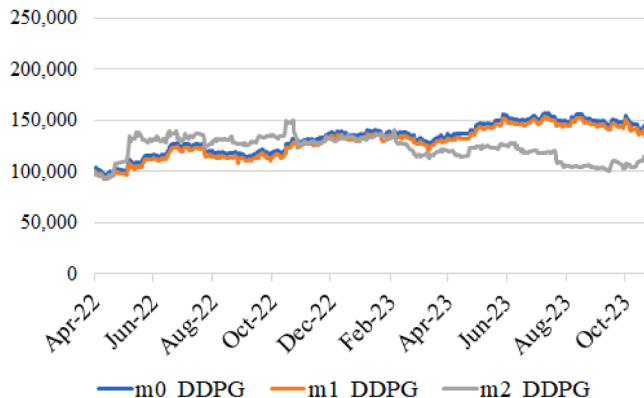


Fig. 9. TAV movement of DDPG models.

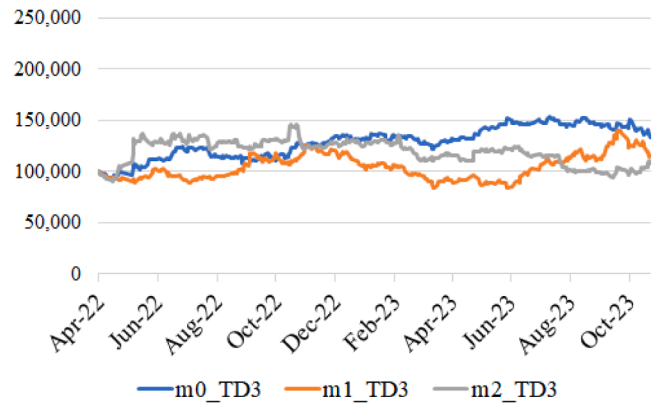


Fig. 11. TAV movement of TD3 models.

performance and, in some cases, can lead to overfitting or reduced risk-adjusted returns.

5. Conclusion

This study explored the impact of varying hidden layer configurations in reinforcement learning models on stock portfolio optimization, focusing on four key algorithms: A2C, DDPG, PPO, and TD3. The results demonstrate that A2C and DDPG perform best with zero hidden layers, achieving solid returns and stable risk management. At the same time, PPO benefits from introducing one hidden layer, significantly improving its performance. However, increasing model complexity further by adding a second hidden layer leads to performance declines, as seen with PPO and DDPG, indicating potential overfitting or diminished adaptability. TD3, on the other hand, remains stable across configurations, showing robustness to changes in model architecture. The study's findings suggest that simpler reinforcement learning models offer more stable returns and better risk-adjusted performance in volatile markets, reducing the risk of overfitting. These insights imply that algorithmic traders could benefit from adapting model complexity based on market conditions, using deeper models in stable markets and more straightforward configurations in dynamic, rapidly changing environments.

For the academic field, this research contributes to the evolving understanding of portfolio management theory by identifying optimal neural architectures within reinforcement learning, advancing AI's role in this domain. For practical investment management, these findings support the development of AI-based portfolio management systems, such as robo-trading and advisory platforms, which can leverage adaptive model architectures to better align with varying market conditions.

This study has certain limitations. It is constrained to a specific set of 45 actively traded stocks in the Indonesian market, which may limit the

generalizability of the results to other markets or asset classes. Additionally, the models are only tested with zero, one, and two hidden layers; future research could explore the impact of other alternative neural network structures. Future research could also include applying these models in more diverse market conditions and include other financial instruments, such as bonds and cryptocurrency, or combining reinforcement learning with machine learning techniques to enhance portfolio optimization strategies further.

ORCID iD: 0009-0004-0931-7415 ORCID record is <https://orcid.org/0009-0004-0931-7415>

Author agreement statement

We the undersigned declare that this manuscript is original, has not been published before and is not currently being considered for publication elsewhere. We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who satisfied the criteria for authorship but are not listed. We further confirm that the order of authors listed in the manuscript has been approved by all of us. We understand that the Corresponding Author is the sole contact for the Editorial process. He/she is responsible for communicating with the other authors about progress, submissions of revisions and final approval of proofs.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Abolmakarem, S., Abdi, F., Khalili-Damghani, K., & Didekhani, H. (2024). A multi-stage machine learning approach for stock price prediction: Engineered and derivative indices. *Intelligent Systems with Applications*, 24, Article 200449. <https://doi.org/10.1016/j.iswa.2024.200449>
- Aboussalah, A., Xu, Z., & Lee, C. (2020). What is the value of the cross-sectional approach to deep reinforcement learning? *Quantitative Finance*, 22, 1091–1111. <https://doi.org/10.1080/14697688.2021.2001032>
- Alkhatib, K., Khazaleh, H., Alkhazaleh, H., Alsoud, A., & Abualigah, L. (2022). A new stock price forecasting method using active deep learning approach. *Journal of Open Innovation: Technology, Market, and Complexity*. <https://doi.org/10.3390/joitmc8020096>
- Althelaya, K., Mohammed, S., & El-Alfy, E. (2021). Combining deep learning and multiresolution analysis for stock market forecasting. *IEEE Access: Practical innovations, open solutions*, 9, 13099–13111. <https://doi.org/10.1109/ACCESS.2021.3051872>
- Alonso, M., & Srivastava, S. (2020). Deep reinforcement learning for asset allocation in US equities. *CompSciRN: Other Machine Learning (Topic)*. <https://doi.org/10.2139/ssrn.3711487>
- Baca, C., & Chen, W. (2021). Deep reinforcement learning for portfolio management of markets with a dynamic number of assets. *Expert Syst. Appl.*, 164, Article 114002. <https://doi.org/10.1016/j.eswa.2020.114002>
- Bai, Z., Zhao, Y., Zhou, Z., Li, W., Gao, Y., Tang, Y., Dai, L., & Dong, Y. (2023). Mercury: A deep reinforcement learning-based investment portfolio strategy for risk-return balance. *IEEE access: practical innovations, open solutions*, 11, 78353–78362. <https://doi.org/10.1109/ACCESS.2023.3298562>
- Esteso, A., Peidro, D., Mula, J., & Díaz-Madroño, M. (2022). Reinforcement learning applied to production planning and control. *International Journal of Production Research*, 61, 5772–5789. <https://doi.org/10.1080/00207543.2022.2104180>
- Eng, P. (2022). Securities trading in an emerging market: Indonesia, 1890s–1940s. *Financial History Review*, 29, 219–246. <https://doi.org/10.1017/S0968565022000099>
- Fan, J., & Peng, S. (2022). Financial stock investment management using deep learning algorithm in the internet of things. *Computational Intelligence and Neuroscience*. <https://doi.org/10.1155/2022/4514300>, 2022.
- Ferreira, F., Gandomi, A., & Cardoso, R. (2021). *Artificial Intelligence Applied to Stock Market Trading: A Review*. *IEEE Access*, 9, 30898–30917. <https://doi.org/10.1109/ACCESS.2021.3058133>
- Fiess, N., & MacDonald, R. (2002). Towards the fundamentals of technical analysis: Analysing the information content of high, low and close prices. *Economic Modelling*, 19, 353–374. [https://doi.org/10.1016/S0264-9993\(01\)00067-0](https://doi.org/10.1016/S0264-9993(01)00067-0)
- Goluz, S., Bauman, T., Kovačević, T., & Kostanjcar, Z. (2023). Imitation Learning for Financial Applications. In *2023 46th MIPRO ICT and Electronics Convention (MIPRO)* (pp. 1130–1135). <https://doi.org/10.23919/MIPRO57284.2023.10159778>
- Hashmi, S., Gilal, M., & Wong, W. (2021). Sustainability of Global Economic Policy and Stock Market Returns in Indonesia. *Sustainability*, 13, 5422. <https://doi.org/10.3390/SU13105422>
- Hao, Y., & Gao, Q. (2020). Predicting the trend of stock market index using the hybrid neural network based on multiple time scale feature learning. *Applied Sciences*. <https://doi.org/10.3390/app10113961>
- Hieu, L. (2020). Deep reinforcement learning for stock portfolio optimization. *ArXiv, abs/2012.06325*. [10.7763/IJMO.2020.V10.761](https://doi.org/10.7763/IJMO.2020.V10.761)
- Huang, H., Gao, T., Li, P., Guo, J., Zhang, P., & Du, N. (2023). Model-based reinforcement learning with non-Gaussian environment dynamics and its application to portfolio optimization. *Chaos (Woodbury, N.Y.)*, 33, 8. <https://doi.org/10.1063/5.0155574>
- Huang, Z., & Tanaka, F. (2021). MSPM: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management. *PLoS ONE*, 17. <https://doi.org/10.1371/journal.pone.0263689>
- Jiang, C., & Wang, J. (2022). A Portfolio Model with Risk Control Policy Based on Deep Reinforcement Learning. *Mathematics*. <https://doi.org/10.3390/math11010019>
- Kuo, Chia-Hsuan, Chen, Chiao-Ting, Lin, Sin-Jing, & Huang, Szu-Hao (2021). Improving Generalization in Reinforcement Learning-Based Trading by Using a Generative Adversarial Market Model. In *IEEE Access*. <https://doi.org/10.1109/ACCESS.2021.3068269>, 1–1.
- Kong, M., & So, J. (2023). Empirical Analysis of Automated Stock Trading Using Deep Reinforcement Learning. *Applied Sciences*. <https://doi.org/10.3390/app13010633>
- Lee, N., & Moon, J. (2023). Offline reinforcement learning for automated stock trading. *IEEE access: practical innovations, open solutions*, 11, 112577–112589. <https://doi.org/10.1109/ACCESS.2023.3324458>
- Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance (arXiv:<https://doi.org/10.48550/arXiv.2011.09607>)
- Liu, F., Li, Y., Li, B., Li, J., & Xie, H. (2021). Bitcoin transaction strategy construction based on deep reinforcement learning. *Applied Soft Computing*. <https://doi.org/10.1016/j.asoc.2021.107952>
- Mattera, G., & Mattera, R. (2023). Shrinkage estimation with reinforcement learning of large variance matrices for portfolio selection. *Intelligent Systems with Applications*, 17, Article 200181. <https://doi.org/10.1016/j.iswa.2023.200181>
- Meng, T., & Khushi, M. (2019). Reinforcement learning in financial markets. *Data*, 4, 110. <https://doi.org/10.3390/DATA4030110>
- Mughal, A. H., Chadalavada, P., Munir, A., Dutta, A., & Qureshi, M. A. (2022). Design of deep neural networks for transfer time prediction of spacecraft electric orbit-raising. *Intelligent Systems with Applications*, 15, Article 200092. <https://doi.org/10.1016/j.iswa.2022.200092>
- Olschewski, S., Diao, L., & Rieskamp, J. (2021). Reinforcement learning about asset variability and correlation in repeated portfolio decisions. *Journal of Behavioral and Experimental Finance*. <https://doi.org/10.1016/j.jbef.2021.100559>
- Rath, S., Sahu, B., & Nayak, M. (2021). A novel stock index prediction approach by combining extreme learning machine with symbiosis organisms search algorithm. *International Journal of Social Ecology and Sustainable Development*, 12, 21–47. <https://doi.org/10.4018/IJSESD.2021040102>
- Sagiraju, H., & Mogalla, S. (2021). Application of multilayer perceptron to deep reinforcement learning for stock market trading and analysis. *Indonesian Journal of Electrical Engineering and Computer Science*. <https://doi.org/10.11591/ijeecs.v24.i3.pp1759-1771>
- Santos, G., Garruti, D., Barboza, F., Souza, K., Domingos, J., & Veiga, A. (2023). Management of investment portfolios employing reinforcement learning. *PeerJ Computer Science*. <https://doi.org/10.7717/peerj-cs.1695>
- Siska, E., Duraipandi, O., & Widodo, P. (2023). Determinants of Indonesian stock market development: Implementation of an ARDL bound testing approach. *Investment Management and Financial Innovations*. [https://doi.org/10.21511/imfi.20\(4\).2023.07](https://doi.org/10.21511/imfi.20(4).2023.07)
- Sreenivas, N. K., & Rao, S. (2022). Safe deployment of a reinforcement learning robot using self stabilization. *Intelligent Systems with Applications*, 16, Article 200105. <https://doi.org/10.1016/j.iswa.2022.200105>
- Tan, H. (2021). Reinforcement learning with deep deterministic policy gradient. In *2021 International Conference on Artificial Intelligence, Big Data and Algorithms (CAIBDA)* (pp. 82–85). <https://doi.org/10.1109/CAIBDA53561.2021.00025>
- Weng, L., Sun, X., Xia, M., Liu, J., & Xu, Y. (2020). Portfolio trading system of digital currencies: A deep reinforcement learning with multidimensional attention gating mechanism. *Neurocomputing*, 402, 171–182. <https://doi.org/10.1016/j.neucom.2020.04.004>
- Zhang, P., & Shen, C. (2019). Choice of the number of hidden layers for back propagation neural network driven by stock price data and application to price prediction. *Journal of Physics: Conference Series*, 1302. <https://doi.org/10.1088/1742-6596/1302/2/022017>